CS695 Assignment 2 - Part 1a

Samar Perwez [22B3913]

February 1, 2025

1 Understanding KVM API in simple-kvm.c

The provided simple-kvm.c file demonstrates the usage of KVM APIs to set up and execute a virtual machine (VM). In this section, we analyze the logical steps involved in initializing and running a VM in protected mode.

1.1 Flowchart

The flowchart below outlines the logical steps to set up and execute a VM using KVM. Each step is labeled and described accordingly.

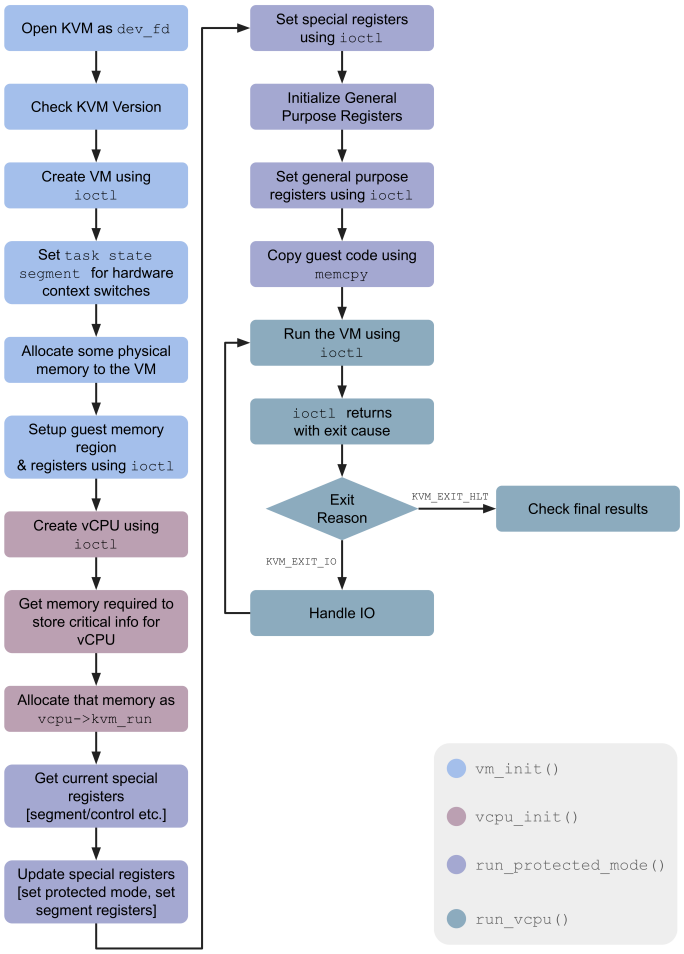

Figure 1: Flowchart for setting up and executing a VM using KVM

2 Code Snippet Explanations

2.1 Snippet (a)

```
extern const unsigned char guest64[], guest64_end[];
```

The given line informs the compiler that in some other external object file there exist two symbols that denote the start and end of a contiguous block of 64-bit guest code. These symbols are defined in the Linker Script (payload.ld),

```
.payload64 0 : AT(LOADADDR(.payload32)+SIZEOF(.payload32)) {
        guest64 = .;
        guest64.img.o
        guest64_end = .;
}
```

These lines tell us that guest64 is set to the current location counter just before the guest binary code. Similarly, guest64_end is set to the end of the guest binary code. SO, when the linker generates an object file, that becomes part of the final linked binary. In the simple-kvm code,

```
memcpy(vm->mem, guest64, guest64_end - guest64);
```

Uses guest64_end and guest64 to compute the total size of the guest code which is to be copied.

2.2 Snippet (b)

```
pml4[0] = PDE64_PRESENT | PDE64_RW | PDE64_USER | pdpt_addr;
pdpt[0] = PDE64_PRESENT | PDE64_RW | PDE64_USER | pd_addr;
pd[0] = PDE64_PRESENT | PDE64_RW | PDE64_USER | PDE64_PS;

sregs->cr3 = pml4_addr;
sregs->cr4 = CR4_PAE;
sregs->cr0 = CR0_PE | CR0_MP | CR0_ET | CR0_NE | CR0_WP | CR0_AM | CR0_PG;
sregs->efer = EFER_LME | EFER_LMA;
```

Here, we are setting up the paging structures required for virtual memory.

The first three lines set up the highest level, PML4, the next level, PDPT and the next level PD page table entries. It marks all of them as present, writable; enables user access and points to the corresponding lower level entry. However, the last level PD, has the PDE64_PS flag enabled indicating that it is directly a 2MB page, removing a need for the lowest level page.

After this, we set up the control registers, CR3 is updated to point to the pml4_addr, the Physical Address Extension flag responsible for long mode and 64-bit addressing is enabled, protected mode and paging is enabled in CR0, finally the Extended Feature Enable Register is updated to enable and activate long mode.

2.3 Snippet (c)

```
vm->mem = mmap(NULL, mem_size, PROT_READ | PROT_WRITE, MAP_PRIVATE |
    MAP_ANONYMOUS | MAP_NORESERVE, -1, 0);

madvise(vm->mem, mem_size, MADV_MERGEABLE);
```

This code snippet deals with allocating pseudo-physical memory for the VM. The first line allocates memory of size `mem_size`, and enables Read/Write access. This memory is not backed to any file, and is private to that process. Further `MAP_NORESERVE` ensures that no swap space is reserved immediately.

The second line advises the kernel that this memory region is a candidate for Kernel Samepage Merging, which reduces memory usage by merging identical pages.

2.4 Snippet (d)

```
case KVM_EXIT_IO:

if (vcpu->kvm_run->io.direction == KVM_EXIT_IO_OUT &&
    vcpu->kvm_run->io.port == 0xE9) {
     char *p = (char *)vcpu->kvm_run;
     fwrite(p + vcpu->kvm_run->io.data_offset,
            vcpu->kvm_run->io.size, 1, stdout);
     fflush(stdout);
     continue;
}
```

This code snippet handles I/O exits from the guest. When the guest writes to port '0xE9', the hypervisor captures the output and prints it to the host's standard output. The VM is then rescheduled

2.5 Snippet (e)

```
memcpy(&memval, &vm->mem[0x400], sz);
```

This code snippet reads `sz` bytes from the guest's memory at offset `0x400` into `memval`. In other words, `0x400` is the pseudo-physical address of the VM which is read from inside the hypervisor using `vm->mem`.